

THE UNIVERSITY OF DODOMA.



COLLEGE OF INFORMATICS AND VIRTUAL EDUCATION.

GROUP 13 PROJECT REPORT.

Asynchronous Timer2 Low-PowerTimekeeping System

MEMBERS.

NAME	REGISTRATION	COURSE
OLARIVU THADAYO	T23-03-18979	Bsc. CE3
FRANK JULIUS	T23-03-12976	Bsc. CE3
NKIYA MANDA	T22-03-08714	Bsc. CE4
KELVIN ASSENGA	T22-03-11783	Bsc. CE3

Abstract

This report documents the design and implementation of a low-power real-time timekeeping system using the ATmega32 microcontroller. The central objective is to demonstrate the asynchronous operation of Timer/Counter2 driven by an external 32.768 kHz watch crystal connected to the dedicated TOSC1 and TOSC2 oscillator pins (PC6 and PC7). By operating Timer2 from a separate low-frequency clock domain, the system achieves one-second timekeeping interrupts while the main CPU spends the majority of its time in Power-Save sleep mode, halting all activity except the asynchronous timer — a design pattern fundamental to battery-operated embedded devices such as alarm clocks, data loggers, and wearable sensors.

The system maintains a real-time clock (HH:MM:SS), drives a periodic wake-up LED on PB0 at ten-second intervals, activates a buzzer on PB1 when a user-programmed alarm time is reached, and accepts user input through three push buttons for time reset, alarm setting, and field increment. Current time and alarm state are transmitted over USART to a serial terminal at 9600 baud. A hardware schematic and PCB layout were designed in KiCad, firmware was developed in MPLAB for VS Code using bare-metal C targeting the AVR architecture, and the system was verified in SimulIDE using an internal-clock fallback path because SimulIDE does not model the external TOSC oscillator circuit.

Keywords: *ATmega32, asynchronous Timer2, TOSC1, TOSC2, 32.768 kHz crystal, real-time clock, low-power design, Power-Save mode, embedded timekeeping, alarm system, AVR.*

Table of Contents

Front Matter

- Title Page
- Abstract
- Table of Contents

Chapter One: Introduction

- 1.1 Background
- 1.2 Problem Statement
- 1.3 Proposed Solution
- 1.4 Project Objectives
- 1.5 Scope and Limitations
- 1.6 Report Organization

Chapter Two: System Design

- 2.1 System Overview and Block Diagram
- 2.2 Pin Mapping and Connection Strategy
- 2.3 Component Selection and Bill of Materials
- 2.4 Timer2 Asynchronous Clock Configuration and Timing Calculations
- 2.5 Power Supply and Decoupling Strategy
- 2.6 Design Trade-offs and Justifications

Chapter Three: Hardware Implementation (KiCad)

- 3.1 Schematic Design
- 3.2 Electrical Rules Check (ERC)
- 3.3 PCB Layout Design
- 3.4 Design Rules Check (DRC)
- 3.5 Output Generation (Gerber Files and BOM)

Chapter Four: Firmware Implementation

- 4.1 Overview and Development Environment
- 4.2 Firmware Architecture
- 4.3 Timer2 Asynchronous Configuration
- 4.4 Timekeeping and Alarm Logic (ISR)

- 4.5 Button Handling and Debouncing
- 4.6 Serial Output (USART)
- 4.7 Sleep Mode and Wake-Up Strategy
- 4.8 SimulIDE Fallback Mode
- 4.9 GitHub Repository Reference

Chapter Five: Simulation and Testing

- 5.1 Simulation Environment
- 5.2 Firmware Loading and Configuration
- 5.3 Test Cases and Results

Chapter Six: Conclusion

- 6.1 Project Summary
- 6.2 Learning Outcomes and Discussion
- 6.3 Future Work

Chapter One

Introduction

1.1 Background

A fundamental requirement of embedded systems across many application domains — including consumer electronics, industrial monitoring, healthcare devices, and environmental sensing — is the ability to maintain accurate time while consuming as little power as possible. Digital wristwatches, alarm clocks, data loggers, and attendance systems all depend on a time base that continues to run accurately even when the main processor is dormant.

The ATmega32 microcontroller addresses this requirement through a dedicated feature of its Timer/Counter2 peripheral: asynchronous operation. Unlike Timer0 and Timer1, which are always driven by the main system clock, Timer2 can be switched to an entirely separate clock domain by setting the AS2 bit in the ASSR register. When this mode is active, Timer2 is clocked from an external oscillator crystal connected to two dedicated pins — TOSC1 (PC6) and TOSC2 (PC7) — completely independently of whatever clock source the CPU is using.

The standard crystal for this purpose is the 32.768 kHz watch crystal, a component found in virtually every battery-powered timekeeping device. Its frequency is 2^{15} Hz, which means it divides down to exactly 1 Hz through purely binary counting — no rounding error, no frequency drift from prescaler mismatch. With the correct Timer2 prescaler setting, the hardware generates one interrupt per second with no software overhead required for the time base itself.

This project introduces students to asynchronous timer operation, real-time clock design, low-power sleep strategies, alarm logic, user input handling, and serial output — a comprehensive introduction to practical embedded system design around a single, elegant hardware feature.

1.2 Problem Statement

Embedded systems deployed in power-constrained environments — battery-operated clocks, remote sensors, attendance terminals — cannot afford to keep the CPU active continuously just to increment a time counter. A microcontroller running at 8 MHz with its

full peripheral set active draws tens of milliamperes; on a small battery, this translates to hours of operation instead of months or years.

Furthermore, timekeeping based on software loops tied to the main system clock is fragile: any interrupt latency, any time spent in non-timekeeping code, or any change to the CPU frequency corrupts the time base. A hardware timer driven by a dedicated, independent crystal eliminates all of these failure modes simultaneously.

The challenge, therefore, is to design a system that maintains accurate time 24 hours a day while keeping the CPU asleep as much as possible, waking only when something requires attention — a new second has elapsed, a button has been pressed, or an alarm condition has been reached.

1.3 Proposed Solution

This project implements a low-power timekeeping and alarm system using the ATmega32's asynchronous Timer2 capability. A 32.768 kHz watch crystal drives Timer2 through the TOSC1/TOSC2 pins, with the prescaler set to /128, producing one Timer2 overflow interrupt every exactly 1 second. The CPU spends almost all of its time in Power-Save sleep mode, which halts the CPU and all synchronous peripherals while keeping the asynchronous Timer2 domain fully active.

Each Timer2 overflow interrupt wakes the CPU for a few microseconds to increment the clock variables, check the alarm condition, toggle the wake-up LED at ten-second intervals, control the buzzer, and set a flag for the main loop to refresh the serial terminal display. The CPU then returns to sleep until the next second interrupt. Three push buttons allow the user to reset the clock to 00:00:00, enter alarm-setting mode, and increment the selected alarm field.

1.4 Project Objectives

The primary objectives of this project are as follows:

- To configure Timer2 in asynchronous mode using an external 32.768 kHz watch crystal on TOSC1 and TOSC2, producing one overflow interrupt per second.
- To implement a real-time software clock (hours, minutes, seconds) maintained entirely within the Timer2 overflow ISR.
- To implement a user-programmable alarm that activates a buzzer for a configurable duration when the current time matches the alarm time.
- To implement a periodic wake-up LED that toggles every ten seconds as a visible heartbeat indicator.

- To implement low-power operation using Power-Save sleep mode, with the CPU waking only on Timer2 overflow.
- To handle user input from three buttons with software debouncing: PA0 (reset), PA1 (set alarm mode), and PA2 (increment).
- To transmit the current time, alarm time, and operating mode to a serial terminal via USART at 9600 baud every second.
- To design the complete circuit schematic and PCB layout in KiCad, following the oscillator placement and noise-separation requirements of the project specification.
- To verify the system in SimulIDE, documenting the simulator limitation regarding TOSC oscillator modelling and demonstrating the equivalent internal-clock fallback.
- To organise all project artefacts in a structured GitHub repository with meaningful commit history.

1.5 Scope and Limitations

Scope:

- Timekeeping resolution of one second, maintained in hardware by Timer2 asynchronous overflow.
- Alarm functionality with buzzer activation for a configurable duration (default 3 seconds).
- Wake-up LED toggle at every ten-second mark as a low-power heartbeat indicator.
- Serial terminal output over USART at 9600 baud, 8N1.
- Software button debouncing with a 20 ms delay-and-confirm strategy.
- SimulIDE-based verification using an internal-clock CTC fallback path for Timer2.

Limitations:

- SimulIDE does not model the external 32.768 kHz TOSC oscillator circuit. The simulator fallback uses an internal-clock CTC configuration on Timer2 with SLEEP_MODE_IDLE to demonstrate identical logic. This limitation is documented in the firmware header and in Chapter 5.
- The system does not include a battery backup for the clock registers; time is lost on power cycle.
- No display other than the USART serial terminal is implemented; an LCD or 7-segment display would require additional GPIO.
- The buzzer is driven directly from PB1 without a transistor or MOSFET driver; this is acceptable for a simulation buzzer model but a real piezo buzzer at higher current would require a driver stage.
- Physical hardware prototype and oscilloscope verification are outside the scope of this report; all verification is simulation-based.

1.6 Report Organization

The remainder of this report is structured as follows:

- Section 2: System Design presents the block diagram, pin mapping, bill of materials, Timer2 timing calculations, and power strategy.
- Section 3: Hardware Implementation describes the KiCad schematic, ERC, PCB layout, DRC, and Gerber file generation.
- Section 4: Firmware Implementation explains the asynchronous timer configuration, ISR logic, sleep strategy, button handling, USART output, and SimulIDE fallback.
- Section 5: Simulation and Testing documents the SimulIDE setup, firmware loading, and all test case results.
- Section 6: Conclusion summarises outcomes, learning points, and suggested future extensions.

Chapter Two: System Design

2.1 System Overview and Block Diagram

The system is organized into six functional blocks that form a pipeline from the external time reference through to user-visible outputs:

- Clock Source Block — 32.768 kHz watch crystal on TOSC1/TOSC2, providing an independent asynchronous clock to Timer2.
- Timer Block — ATmega32 Timer2 in asynchronous overflow mode; overflows every 1 second and fires the timekeeping ISR.
- Control Block — ATmega32 CPU running at 8 MHz internal RC, spending most time in Power-Save sleep; wakes on Timer2 overflow.
- Output Block — PB0 (wake-up LED via 330 Ω resistor) and PB1 (buzzer) driven directly from the ISR and main loop logic.
- User Input Block — Three active-LOW push buttons on PA0 (reset), PA1 (set alarm), and PA2 (increment), read with software debouncing.
- Communication Block — USART TX on PD1 at 9600 baud, 8N1, streaming current time and mode to a serial terminal.

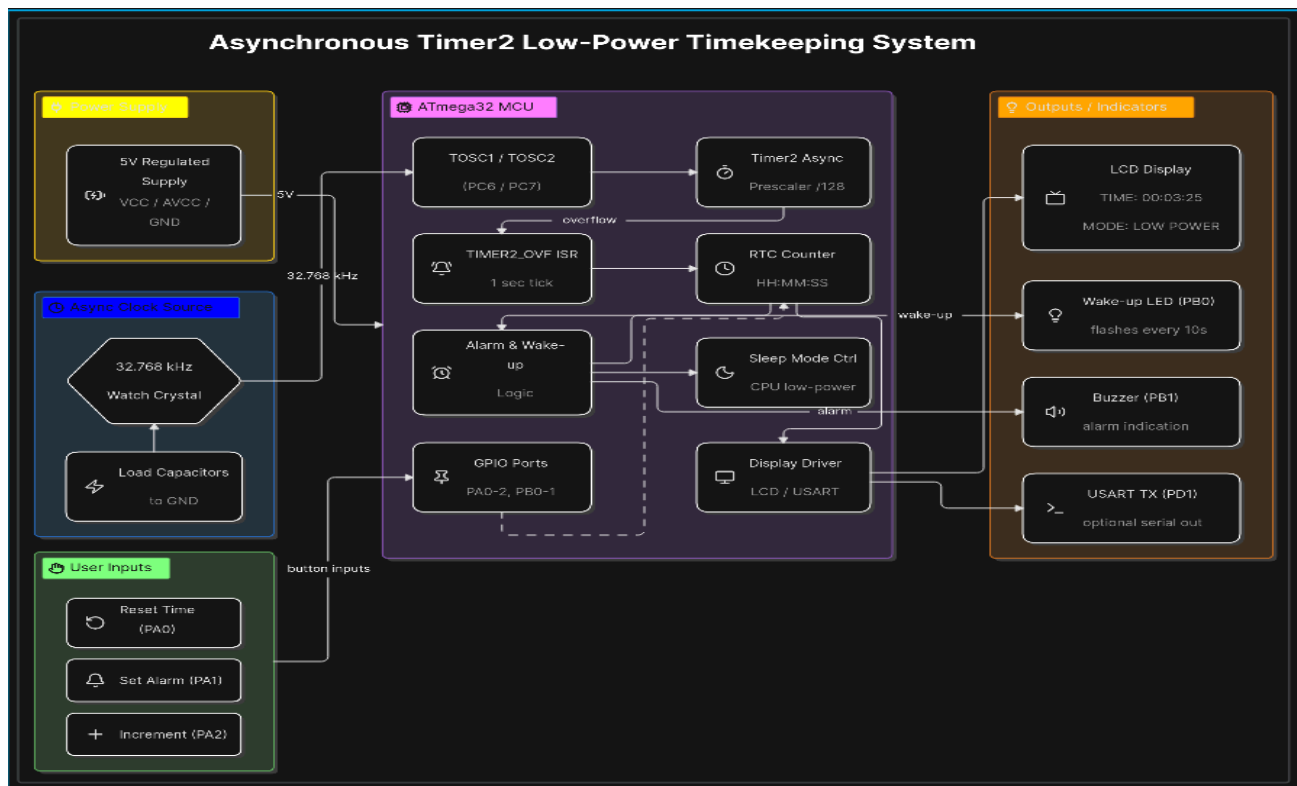


Figure 2.1 — High-level system block diagram showing data flow from crystal to CPU to outputs

2.2 Pin Mapping and Connection Strategy

2.2.1 ATmega32 Pin Allocation

Table 2.1 provides the complete pin mapping between the ATmega32 and all connected devices.

ATmega32 Pin	Pin Name / Signal	Direction	Connected Device	Function
PC6 (Pin 28)	TOSC1	Input	32.768 kHz Crystal	Timer2 async clock input
PC7 (Pin 29)	TOSC2	Output	32.768 kHz Crystal	Timer2 async oscillator return
PB0 (Pin 1)	GPIO	Output	LED via 330 Ω	Periodic wake-up LED
PB1 (Pin 2)	GPIO	Output	Buzzer	Alarm buzzer output
PA0 (Pin 40)	GPIO	Input	Push button SW1	Reset clock to 00:00:00
PA1 (Pin 39)	GPIO	Input	Push button SW2	Cycle alarm-set mode
PA2 (Pin 38)	GPIO	Input	Push button SW3	Increment selected alarm field
PD1 (Pin 15)	TXD	Output	Serial terminal	USART TX, 9600 baud, 8N1
Pin 9 (RST)	RESET	Input	SW4 + 10 k Ω pull-up	Hardware microcontroller reset
Pin 10	VCC	Power	+5 V supply	Digital power
Pin 30	AVCC	Power	+5 V supply	Analog power domain
Pin 32	AREF	Input	100 nF to GND	ADC reference decoupling
Pins 11, 31	GND	Power	0 V reference	Ground

Table 2.1 — Complete ATmega32 pin allocation

2.3 Component Selection and Bill of Materials

Table 2.2 lists all hardware components required to build the system.

Item	Component	Qty	Value / Model	Purpose	Notes
1	ATmega32 MCU	1	ATmega32 A-PU	Main microcontroller	40-pin DIP
2	Watch crystal	1	32.768 kHz	Timer2 async source	TOSC1/TOSC2
3	Load capacitor	2	12 pF ceramic	Crystal oscillator support	C1, C2
4	Decoupling cap	2	100 nF ceramic	Power filtering	C3 (AREF), C4 (VCC/AVCC)
5	LED	1	5 mm any colour	Wake-up indicator	PB0 via R1
6	Current-limit R	1	330 Ω	LED protection	R1
7	Pull-up resistor	1	10 k Ω	RESET pin pull-up	R2
8	Buzzer	1	Passive piezo	Alarm output	PB1
9	Push button	4	Tactile SPST	User input + HW reset	SW1–SW4
10	Serial connector	1	2-pin header	USART TX access	J1
11	5 V power supply	1	USB or adapter	System power	≥ 100 mA

Table 2.2 — Bill of Materials (BOM)

2.4 Timer2 Asynchronous Clock Configuration and Timing Calculations

2.4.1 External Crystal Frequency

The 32.768 kHz watch crystal was selected for three reasons: (1) it is the standard frequency used in all wristwatch and RTC ICs; (2) its frequency equals 2^{15} , which means it divides down to exactly 1 Hz through purely binary prescaling — no remainder, no

accumulated error; and (3) its low frequency means it draws minimal current from the crystal oscillator circuit, contributing to the low-power design goal.

2.4.2 Prescaler and Overflow Period Calculation

Timer2 is an 8-bit timer, meaning it counts from 0 to 255 (256 counts per cycle) before overflowing and resetting. The required prescaler value is derived as follows:

```
f_TOSC = 32,768 Hz
Target = 1 overflow per second

Prescaler N such that: (256 × N) / 32768 = 1 second

N = 32768 / 256 = 128

Verification: T_overflow = (256 × 128) / 32768 = 32768 / 32768 = 1.000
second ✓
```

The prescaler /128 corresponds to the CS2[2:0] = 0b101 setting in TCCR2. This configuration produces one Timer2 overflow interrupt every exactly 1 second, with zero software accumulation required.

2.4.3 Asynchronous Register Update Synchronisation

Writing to Timer2 registers (TCNT2, OCR2, TCCR2) while AS2 is set requires waiting for the asynchronous domain to latch the new values. The ASSR register provides busy flags (TCN2UB, OCR2UB, TCR2UB) that must all clear before the CPU continues. Failing to wait can corrupt the register values. The firmware's timer2_init() function implements this mandatory wait loop as required by the ATmega32 datasheet.

Register	Value	Effect
ASSR	AS2 = 1	Select external TOSC oscillator for Timer2
TCNT2	0x00	Clear timer counter before starting
TCCR2	CS22 CS20 (= 0b00000101)	Prescaler /128; normal (overflow) mode
TIMSK	TOIE2 = 1	Enable Timer2 overflow interrupt

Table 2.3 — Timer2 asynchronous register configuration

2.5 Power Supply and Decoupling Strategy

All logic runs from a single 5 V supply. The ATmega32 draws approximately 8–12 mA at 8 MHz in active mode but drops to roughly 1 μ A (CPU halted) plus $\sim$ 15 μ A (Timer2 async oscillator active) during Power-Save sleep. The average current is therefore dominated by the brief 1-second active windows and the asynchronous oscillator, making the overall system suitable for coin-cell or small AAA battery operation on real hardware.

Capacitor	Value	Placement	Purpose
C1	12 pF	TOSC1 pin to GND	Crystal load capacitor
C2	12 pF	TOSC2 pin to GND	Crystal load capacitor
C3	100 nF	AREF pin to GND (close)	ADC reference decoupling
C4	100 nF	VCC/AVCC pins to GND	High-frequency power bypass

Table 2.4 — Decoupling and oscillator load capacitor placement

2.6 Design Trade-offs and Justifications

Decision	Alternative Considered	Rationale for Selected Approach
32.768 kHz external crystal on TOSC1/TOSC2	Software counter on main 8 MHz timer	Hardware timer is immune to interrupt latency; delivers exactly 1 s with no drift. Software counters accumulate error over hours.
Power-Save sleep mode	Idle or no sleep	Power-Save keeps only Timer2 async and USART active; CPU and all synchronous clocks halt. Lowest power compatible with 1-second wake cycles.
Active-LOW buttons with internal pull-ups	External pull-down resistors	Internal pull-ups eliminate four external resistors; active-LOW is the standard for ATmega I/O ports.
Software debounce (20 ms delay + re-read)	Hardware RC filter per button	Simpler circuit; 20 ms delay is imperceptible to users and adequate for mechanical switches.
USART serial terminal output	LCD or 7-segment display	Saves GPIO; any terminal program can display formatted text. Extension to LCD is straightforward by adding rows to <code>update_display()</code> .
330 Ω LED resistor on 5 V	220 Ω or 470 Ω	330 Ω gives ~8.5 mA — clearly visible yet well within PB0 pin source current limit of 40 mA.
8 MHz internal RC oscillator for CPU	External crystal on XTAL1/XTAL2	Frees the crystal pins; UBRR = 51 at 9600 baud gives < 0.2 % baud error at 8 MHz. Accurate USART timing verified in simulation.

Table 2.5 — Key design decisions and justifications

Chapter Three

Hardware Implementation (KiCad Schematic and PCB Layout)

This chapter documents the complete hardware design of the asynchronous Timer2 timekeeping system as implemented in KiCad EDA software. The schematic defines all electrical connections, and the PCB layout translates those connections into a manufacturable board with specific attention to crystal placement and noise separation as required by the project specification.

3.1 Schematic Design

3.1.1 Schematic Organisation

The schematic is logically divided into the following subsystems:

- Crystal Oscillator Subsystem — Y1 (32.768 kHz) with C1 and C2 (12 pF) connected between TOSC1 (PC6) and TOSC2 (PC7), with net labels TOSC1_32768HZ and TOSC2_32768HZ.
- Microcontroller Subsystem — ATmega32A-P with VCC, AVCC, GND, AREF, RESET, and oscillator connections.
- Reset Circuit — SW4 (tactile) connecting RESET to GND; R2 (10 k Ω) pulling RESET to VCC.
- Output Subsystem — D1 (LED) via R1 (330 Ω) on PB0 (net: WAKE_LED); BZ1 (buzzer) on PB1 (net: ALARM_BUZZER).
- Button Subsystem — SW1 (SET_ALARM_BTN on PA1), SW2 (on PA2, increment), SW3 (on PA0, reset clock), each wired with internal pull-ups enabled in firmware.
- Serial Output Subsystem — J1 (2-pin header) connected to PD1/TXD.
- Power Decoupling — C3 and C4 (100 nF) placed on AREF and VCC/AVCC respectively, with PWR_FLAG symbols on VCC and GND rails.

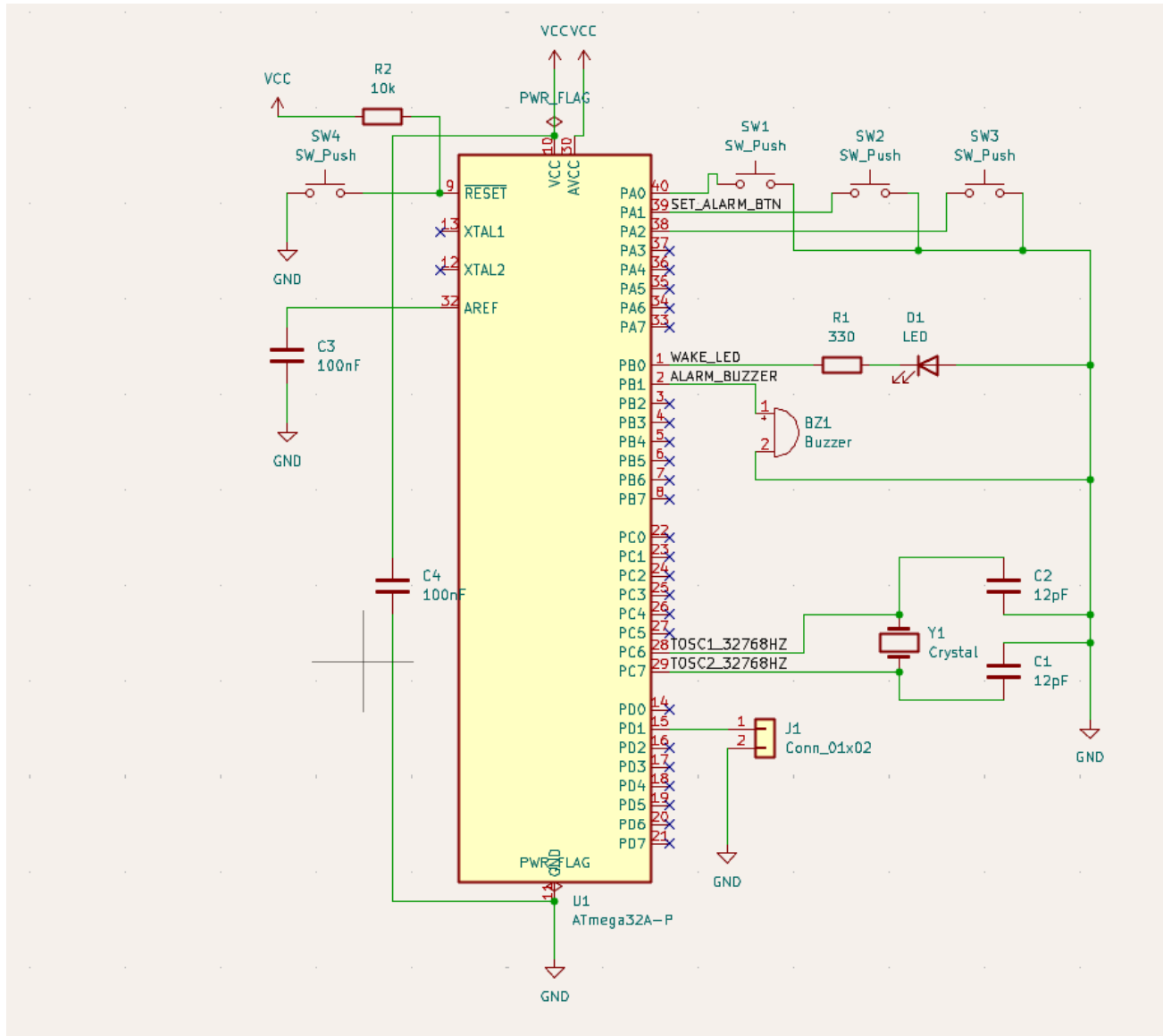


Figure 3.1 — Full system schematic as designed in KiCad Eeschema

3.1.2 Net Labels Used

Net Label	Signal	Source Pin	Destination
TOSC1_32768HZ	Timer2 async clock input	PC6 (Pin 28)	Y1 crystal pin 1
TOSC2_32768HZ	Timer2 async oscillator	PC7 (Pin 29)	Y1 crystal pin 2
WAKE_LED	Wake-up LED drive	PB0 (Pin 1)	R1 → D1 anode
ALARM_BUZZER	Buzzer drive	PB1 (Pin 2)	BZ1 positive terminal
SET_ALARM_BTN	Set alarm button	PA1 (Pin 39)	SW1 lower terminal

Table 3.1 — Required net labels as specified in the project brief

3.2 Electrical Rules Check (ERC)

After completing the schematic, KiCad's Electrical Rules Check (ERC) was performed to identify electrical connectivity errors such as unconnected pins, conflicting drivers, or missing power symbols.

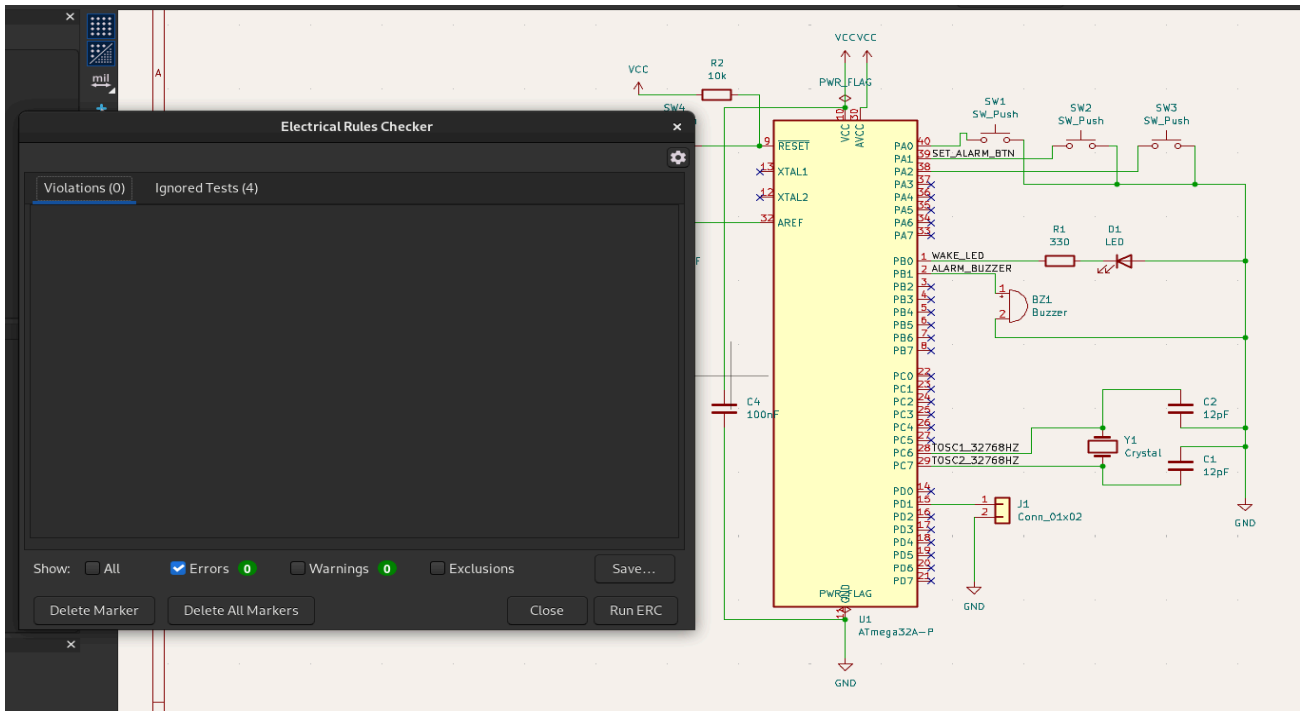


Figure 3.2 — ERC results confirming no electrical errors in the schematic

3.3 PCB Layout Design

The schematic was imported into KiCad PCB layout (Pcbnew) and all components were placed and routed. Key placement decisions were guided by the project specification's oscillator design requirements:

- Y1 (32.768 kHz crystal) was placed as close as physically possible to the PC6/PC7 pins of the ATmega32, minimising trace length on the TOSC1 and TOSC2 nets.
- C1 and C2 (12 pF load capacitors) were placed immediately adjacent to Y1, with short, direct routes to the GND plane.
- The TOSC1/TOSC2 traces were kept short and routed away from PB0, PB1, and any other switching output lines to minimise capacitive coupling and oscillator start-up degradation.
- BZ1 and D1 were placed in an accessible user-facing area. Buzzer and LED drive traces were routed on the opposite side of the board from the crystal to enforce physical noise separation.
- Buttons SW1–SW4 were placed along one edge for easy user access.
- J1 (serial header) was placed on the board edge for simple cable connection.

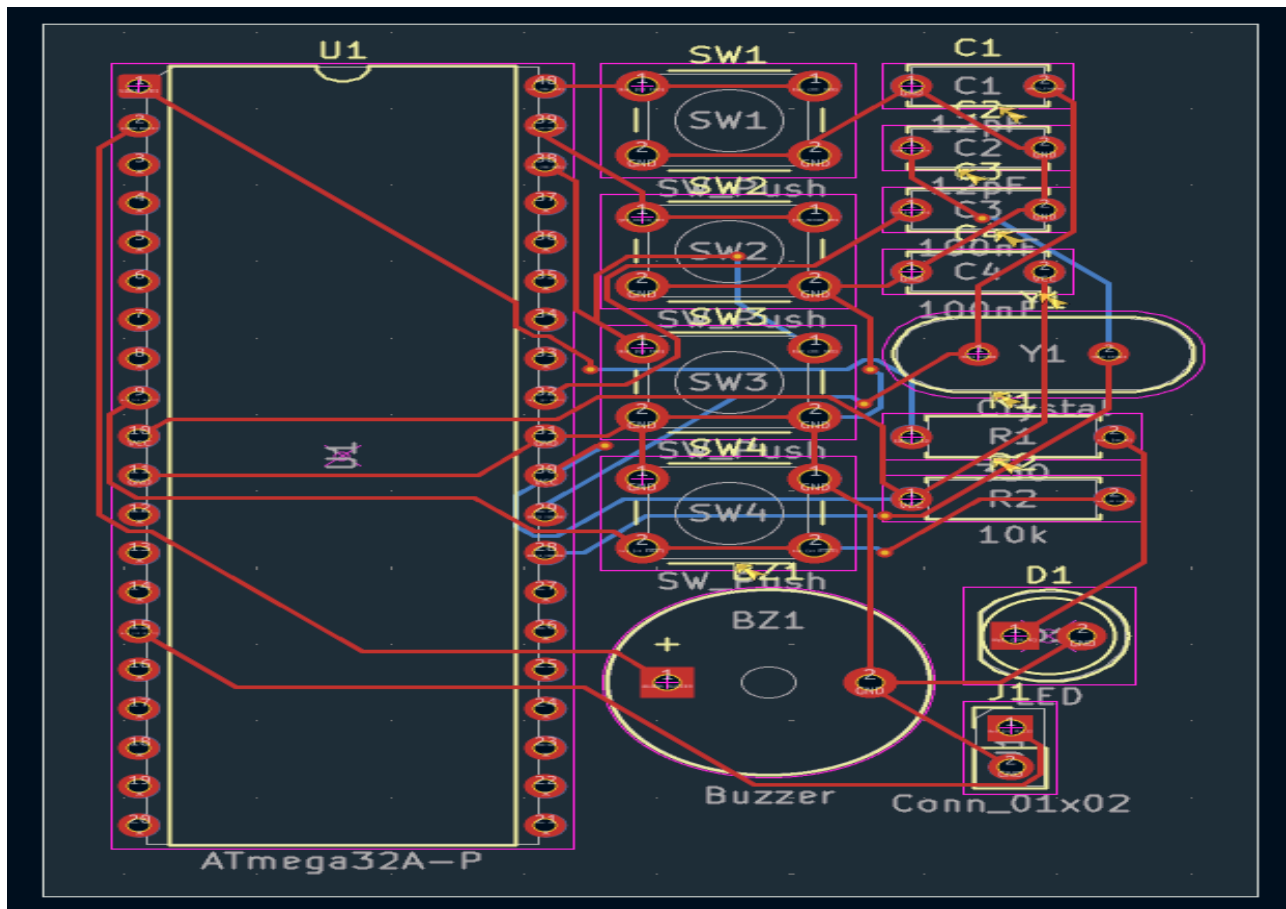


Figure 3.3 — PCB layout showing component placement and copper routing

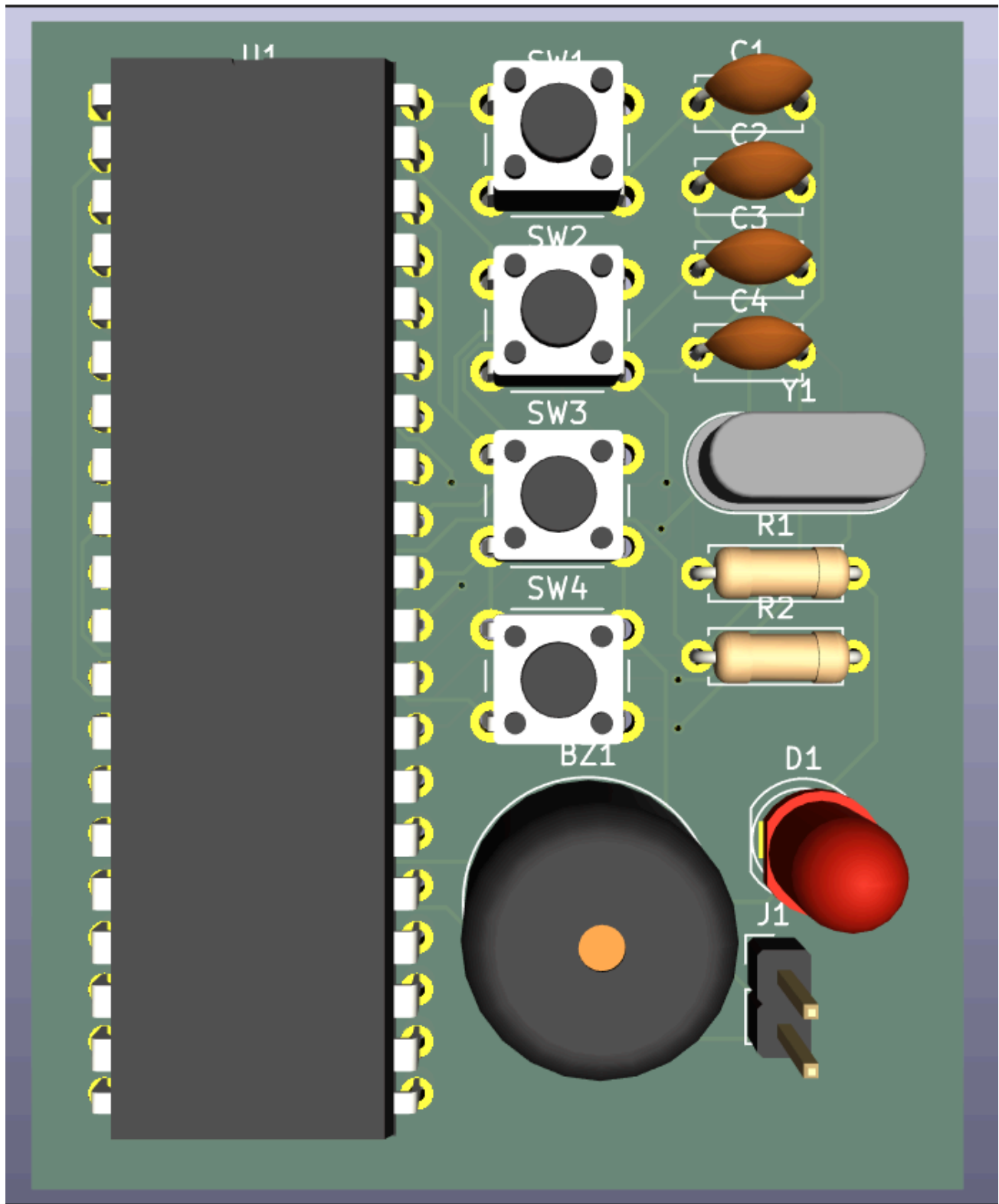


Figure 3.4 — KiCad 3D render of the completed PCB

3.4 Design Rules Check (DRC)

After completing the routing, KiCad's Design Rules Check (DRC) was run to verify that all traces, clearances, and footprint placements comply with the design rules specified for the board.

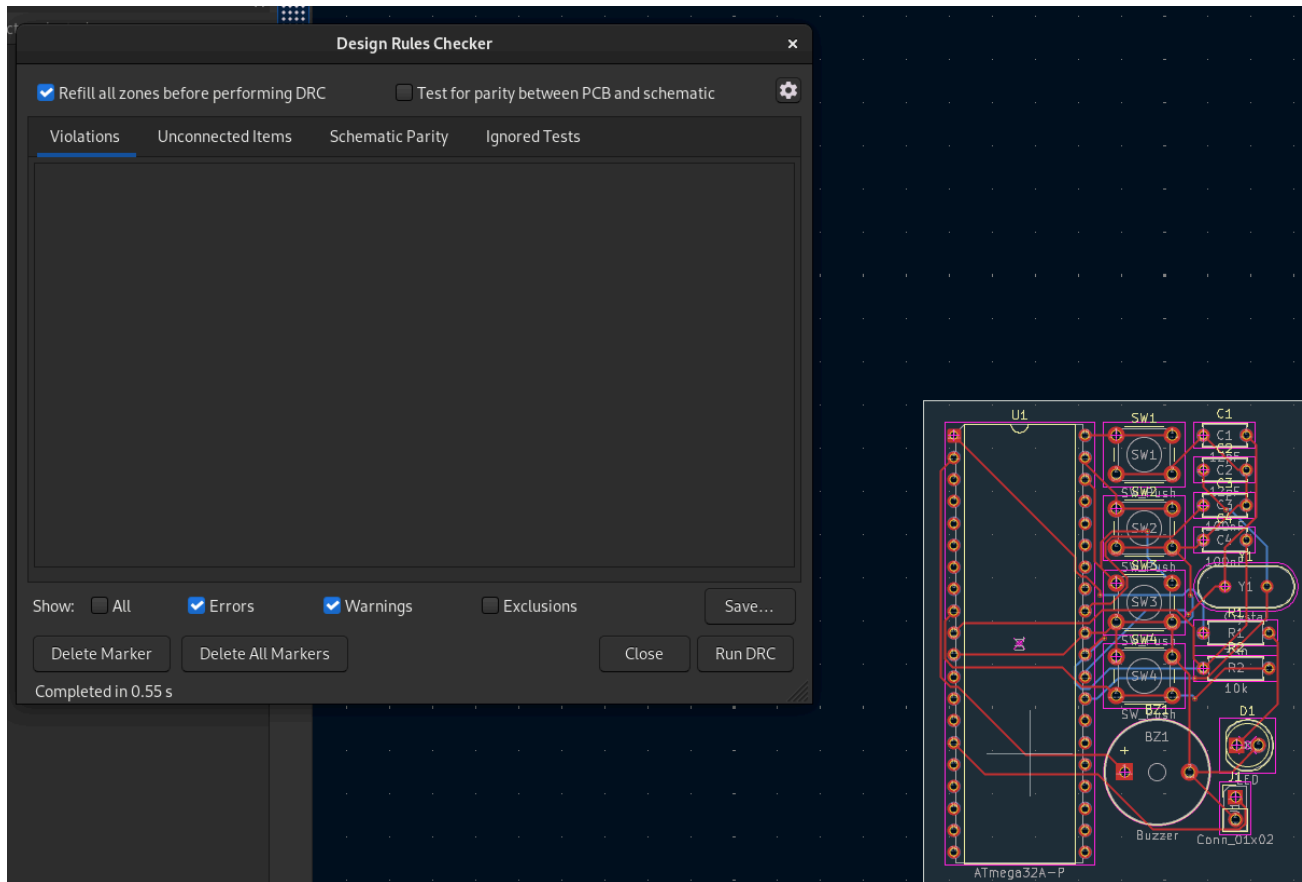


Figure 3.5 — DRC parameters and results confirming no violations

3.5 Output Generation

3.5.1 Gerber File Generation

Gerber files were generated from the completed PCB layout for manufacturing submission. Table 3.2 lists the files produced.

Layer	File Extension	Contents
Top copper	.GTL	Signal traces, pads, vias on component side
Bottom copper	.GBL	Bottom copper / ground plane
Top solder mask	.GTS	Solder mask openings on top layer
Bottom solder mask	.GBS	Solder mask openings on bottom layer
Top silkscreen	.GTO	Component outlines, reference designators, labels
Board outline	.GKO	PCB perimeter / edge cuts
Drill file	.TXT	NC drill coordinates for all holes and vias

Table 3.2 — Gerber file listing for PCB manufacturing

Chapter Four

Firmware Implementation

4.1 Overview and Development Environment

The firmware was developed using MPLAB for VS Code with the AVR-GCC toolchain, targeting the ATmega32 in bare-metal C. No RTOS or middleware libraries were used; all peripheral configuration, interrupt handling, and timing logic is implemented directly using the ATmega32 register set and AVR-libc headers (`avr/io.h`, `avr/interrupt.h`, `avr/sleep.h`, `util/delay.h`).

Parameter	Setting	Notes
MCU	ATmega32	-mmcu=atmega32
CPU Frequency	8 MHz	F_CPU = 8000000UL; internal RC oscillator
Optimisation	-Os	Size-optimised; ISR code is compact
Compiler	AVR-GCC via MPLAB	MPLAB for VS Code extension
Real hardware target	make	Uses TOSC async path and Power-Save sleep
SimulIDE target	make simulide	Defines -DSIMULIDE_MODE; uses CTC + Idle sleep

Table 4.1 — Firmware build environment

4.2 Firmware Architecture

The firmware follows a simple event-driven superloop architecture. The main loop polls buttons and checks for the `g_time_updated` flag set by the ISR; when the flag is set, the display is refreshed and the CPU returns to sleep. All time-critical operations (clock increment, alarm check, LED toggle, buzzer control) occur inside the ISR.

The high-level structure is:

```
main()
{
    gpio_init();           // configure PB0/PB1 outputs, PA0-2 inputs with
pull-ups
    usart_init();          // 9600 baud, 8N1, TX only on PD1
    timer2_init();         // async 32.768 kHz / 128 → 1-second overflow ISR
    sei();                 // enable global interrupts
    update_display();      // print initial state to terminal

    while (1)
    {
        handle_buttons(); // poll PA0, PA1, PA2 with debounce
        if (g_time_updated) // flag set by ISR each second
            update_display(); // print TIME/ALRM/MODE over USART
        enter_sleep();      // Power-Save; wakes on Timer2 overflow
    }
}
```

4.3 Timer2 Asynchronous Configuration

The `timer2_init()` function implements the mandatory five-step procedure for enabling asynchronous Timer2 as specified in the ATmega32 datasheet:

1. Set AS2 bit in ASSR to connect Timer2 to the TOSC oscillator pins.
2. Clear TCNT2 to ensure a clean start.
3. Write TCCR2 with prescaler /128 (CS22 | CS20 = 0b101).
4. Wait until all ASSR busy flags clear (TCN2UB, OCR2UB, TCR2UB), confirming the async domain has latched the new register values. This step is mandatory; skipping it corrupts the timer configuration.
5. Enable the Timer2 overflow interrupt via TOIE2 in TIMSK.

The wait loop in step 4 is the most important synchronisation detail in the entire firmware. The asynchronous Timer2 register bus crosses two clock domains; any write that is not acknowledged by the async domain before the next write or before sleep entry will be silently discarded by the hardware.

4.4 Timekeeping and Alarm Logic (ISR)

The `TIMER2_OVF_vect` ISR fires every 1 second on real hardware. All of the following operations are performed inside the ISR to guarantee they execute exactly once per second with no possibility of drift:

- Increment `g_seconds`; if 60, reset to 0 and increment `g_minutes`.
- If `g_minutes` reaches 60, reset to 0 and increment `g_hours`.
- If `g_hours` reaches 24, reset to 0 (midnight rollover).
- If `g_seconds` is a multiple of `WAKEUP_INTERVAL` (10), toggle PB0 (wake-up LED) using XOR on `PORTB`.
- If the current time (`g_hours:g_minutes:g_seconds`) matches the alarm time and `g_alarm_ringing` is false, set `g_alarm_ringing = true`, load `g_buzz_countdown` with `ALARM_BUZZ_DURATION` (3 seconds), and assert PB1 (buzzer).
- If `g_alarm_ringing` is true and `g_buzz_countdown` reaches zero, clear `g_alarm_ringing` and deassert PB1.
- Set `g_time_updated = true` to signal the main loop that the display needs refreshing.

All variables modified inside the ISR are declared volatile. The main loop uses `cli()/sei()` pairs when reading multi-byte time values to prevent torn reads.

4.5 Button Handling and Debouncing

Buttons are polled in `handle_buttons()` on every main loop iteration. Three logical buttons are handled:

- PA0 (Reset): resets `g_seconds`, `g_minutes`, `g_hours` to zero; clears alarm state; returns mode to `MODE_CLOCK`.
- PA1 (Set Alarm): cycles the application mode through `MODE_CLOCK` → `MODE_SET_ALARM_H` → `MODE_SET_ALARM_M` → `MODE_SET_ALARM_S` → `MODE_CLOCK`.
- PA2 (Increment): increments the alarm field selected by the current mode (hours modulo 24, minutes/seconds modulo 60).

Software debouncing uses a read-delay-read strategy: the pin is sampled once; if LOW (pressed), a 20 ms delay is inserted; the pin is sampled again. Only if both reads agree is the press accepted. After a valid press, the code waits for the physical button release and adds a further 20 ms guard before returning.

4.6 Serial Output (USART)

The USART is configured for 9600 baud, 8N1, transmit-only mode. The baud rate register value is:

$$UBRR = (F_CPU / (16 \times BAUD)) - 1 = (8,000,000 / (16 \times 9600)) - 1 = 51$$

This gives a baud rate error of less than 0.2 %, which is well within the ± 2 % tolerance of standard UART receivers. At 8 MHz with UBRR = 51, the actual baud rate is 9615.4 baud.

The `update_display()` function prints three lines each second using `snprintf` into a 32-byte stack buffer:

```
TIME: HH:MM:SS
ALRM: HH:MM:SS
MODE: LOW POWER (or SET ALARM HH / MM / SS)
---
```

4.7 Sleep Mode and Wake-Up Strategy

The `enter_sleep()` function selects `SLEEP_MODE_PWR_SAVE` and calls `sleep_cpu()` after enabling global interrupts. In Power-Save mode, the ATmega32 halts the CPU, stops all synchronous clocks, and disables all peripherals except the asynchronous Timer2 oscillator. The Timer2 overflow interrupt is the only event that can wake the CPU from this mode.

The wake-up sequence is: Timer2 overflows → interrupt fires → CPU wakes → ISR executes → ISR returns → `sleep_disable()` is called → main loop runs (button check + display update) → `enter_sleep()` called again → CPU sleeps until next second.

This means the CPU is active for only a few hundred microseconds per second, spending the remaining 99.9%+ of its time in the lowest available power state while Timer2 continues ticking from its independent crystal.

4.8 SimulIDE Fallback Mode (SIMULIDE_MODE)

SimulIDE does not simulate the external 32.768 kHz crystal oscillator on TOSC1/TOSC2. If the firmware compiled without the `SIMULIDE_MODE` flag is loaded in the simulator, Timer2 takes the asynchronous path (`AS2 = 1`) and attempts to wait for an oscillator that never starts; the busy flags in `ASSR` never clear, the timer never overflows, and when

`enter_sleep()` is called with `SLEEP_MODE_PWR_SAVE`, the CPU halts permanently because the only possible wake source (Timer2 overflow) is also stalled.

The fallback, enabled by compiling with `make simulide (-DSIMULIDE_MODE)`, configures Timer2 in CTC mode driven by the internal 8 MHz system clock:

```
Prescaler: /1024
OCR2 = 77
f_compare = 8,000,000 / (1024 × 78) ≈ 100.1 Hz ≈ 100 interrupts/second
```

The `TIMER2_COMP_vect` ISR accumulates ticks in `g_tick_count`; when 100 ticks have elapsed, the same timekeeping, alarm, LED, and buzzer logic from the real ISR executes. Because the synchronous system clock now drives Timer2, `SLEEP_MODE_PWR_SAVE` would stop that clock and stall the timer — so the fallback path uses `SLEEP_MODE_IDLE` instead, which halts only the CPU while keeping all timers and peripherals running.

All user-visible behaviour (serial output, button response, alarm, LED) is identical between the two paths. The only differences are the clock source driving Timer2 and the sleep depth.

4.9 GitHub Repository Reference

All project artefacts are maintained in a structured GitHub repository following the layout specified in the project brief:

```
ATmega32_Asynchronous_Timer2_Timekeeping/
├── README.md
├── firmware/
│   ├── src/main.c
│   ├── include/
│   └── hex/
├── kicad/
│   ├── schematic/
│   ├── pcb/
│   └── gerber/
├── simulide/
│   ├── circuit/
│   └── screenshots/
├── documentation/
│   ├── block_diagram.png
│   └── flowchart.png
```

The firmware source is available at: [GitHub Repository URL — insert link here]

Chapter Five

Simulation and Testing

5.1 Simulation Environment

5.1.1 SimulIDE Setup

SimulIDE is a real-time electronic circuit simulator with AVR microcontroller support. Because SimulIDE does not model the external 32.768 kHz TOSC oscillator, the simulation uses the SIMULIDE_MODE firmware build (make simulide), which uses the internal 8 MHz clock to drive Timer2 in CTC mode at approximately 100 ticks per second, accumulating to one logical second in software. This limitation is documented in the firmware header and in Section 4.8; the KiCad schematic remains correct for the actual ATmega32 hardware target.

Parameter	Setting	Purpose
Simulator	SimulIDE v1.0+	AVR circuit simulation and real-time debugging
MCU Model	mega32-2 (ATmega32)	Main controller simulation
MCU Clock Frequency	8 MHz (must match F_CPU)	Required for correct USART baud rate calculation
Firmware	firmware_simulide.hex	SIMULIDE_MODE build; CTC fallback for Timer2
Serial terminal	SerialTerm-8	Receives USART output at 9600 baud, 8N1, ASCII
Sleep mode (sim)	SLEEP_MODE_IDLE	Keeps synchronous Timer2 clock running in sim

Table 5.1 — SimulIDE simulation environment configuration

5.1.2 Simulation Circuit Construction

The simulation circuit in SimulIDE includes the mega32-2 MCU component, a buzzer, an LED with 330 Ω series resistor, three toggle switches (PA0, PA1, PA2), a SerialTerm component connected to PD1 for serial output, and VCC/GND power rails. Note: the TOSC crystal circuit is present in the KiCad schematic but is not included in the SimulIDE circuit because the simulator cannot model it; the firmware fallback makes it unnecessary for simulation purposes.

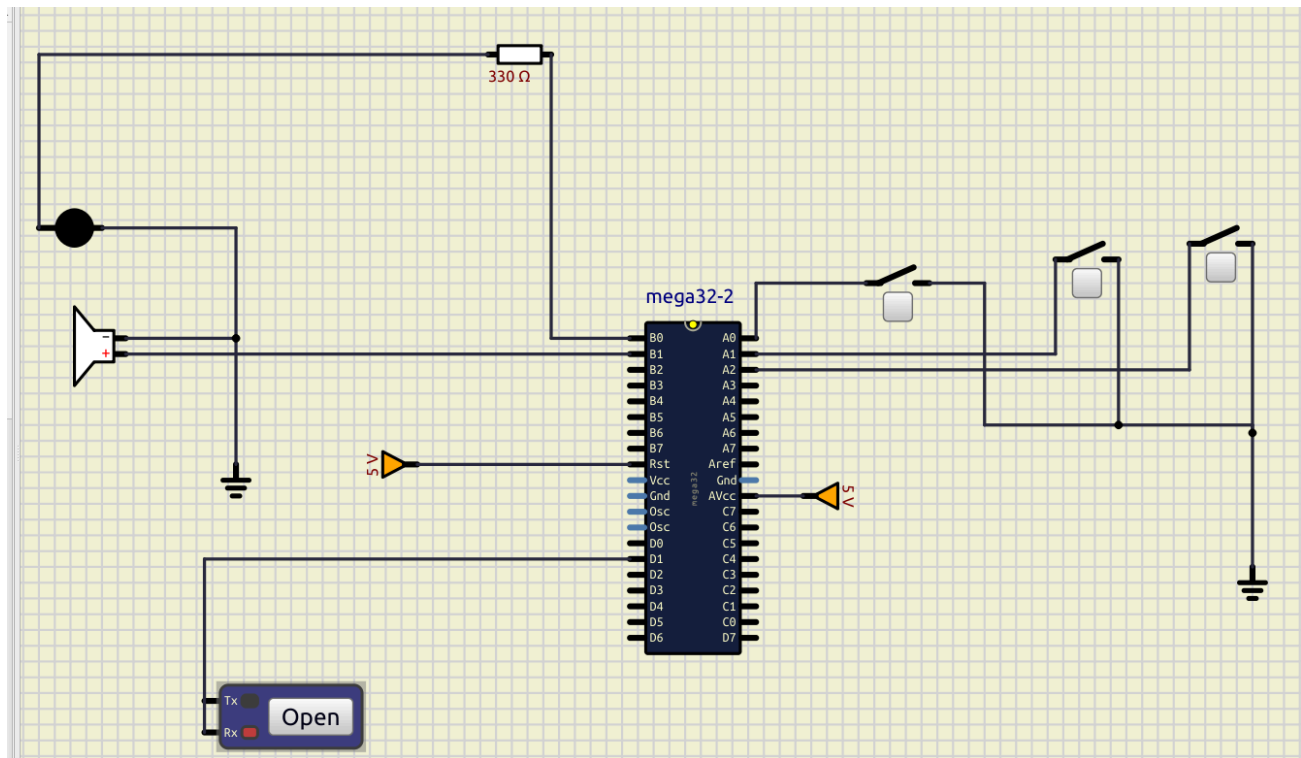


Figure 5.1 — Complete SimulIDE circuit with MCU, buzzer, LED, buttons, and serial terminal

5.2 Firmware Loading and Configuration

5.2.1 HEX File Loading and Clock Configuration

The compiled SimulIDE HEX file was loaded into the simulated ATmega32 using the following procedure:

6. Right-click the mega32-2 component in SimulIDE and select Properties.
7. In the Main tab, set Frequency to 8 and units to MHz. This step is critical: a mismatch between the MCU clock setting and F_CPU causes UBRR to produce the wrong baud rate, resulting in garbled serial output.
8. In the Firmware field, browse to and select firmware/hex/firmware_simulide.hex.
9. Check "Reload hex at Simulation Start" for automatic firmware reloading.

10. Click the check mark to confirm and start the simulation.

Note: During initial development, the MCU clock was set to the default of 16 MHz while the firmware used `F_CPU = 8 MHz`, causing UBRR to be off by a factor of two and producing completely garbled serial output. Setting the clock to 8 MHz resolved this immediately, demonstrating that the baud rate register calculation and the simulator's clock frequency setting must always agree with the `F_CPU` macro in the firmware.

5.3 Test Cases and Results

All test cases specified in the project requirements were executed in SimulIDE. Results are documented below.

TC-01: System Start — Clock Initialises at 00:00:00

Step	Action	Expected Result	Observed Result	Status
1	Start simulation	Boot message printed, TIME: 00:00:00	Boot banner and initial time printed correctly	✓ PASS
2	Observe serial terminal	MODE: LOW POWER displayed	MODE: LOW POWER confirmed	✓ PASS

Table 5.2 — TC-01 Results

Pause

Print: ASCII ▾

Send Text:

CR

Send Value:

```

== ATmega32 Async Timer2
Timekeeping ==
(SimulIDE mode: internal clock
fallback)
Buttons: PA0=Reset
PA1=SetAlarm PA2=Increment
---
TIME:  00:00:00
ALRM:  00:00:00
MODE:  LOW POWER
---
```

Figure 5.2 — Serial terminal showing initial boot message and 00:00:00 start state

TC-02: Timer2 Running — Seconds Increment Every Second

Step	Action	Expected Result	Observed Result	Status
1	Let simulation run	Seconds value increases each second	TIME updates 00:00:25 → 00:00:26 → ... confirmed	✓ PASS
2	Verify timing	One increment per simulation second	Increments align with simulation clock (1 s intervals)	✓ PASS

Table 5.3 — TC-02 Results

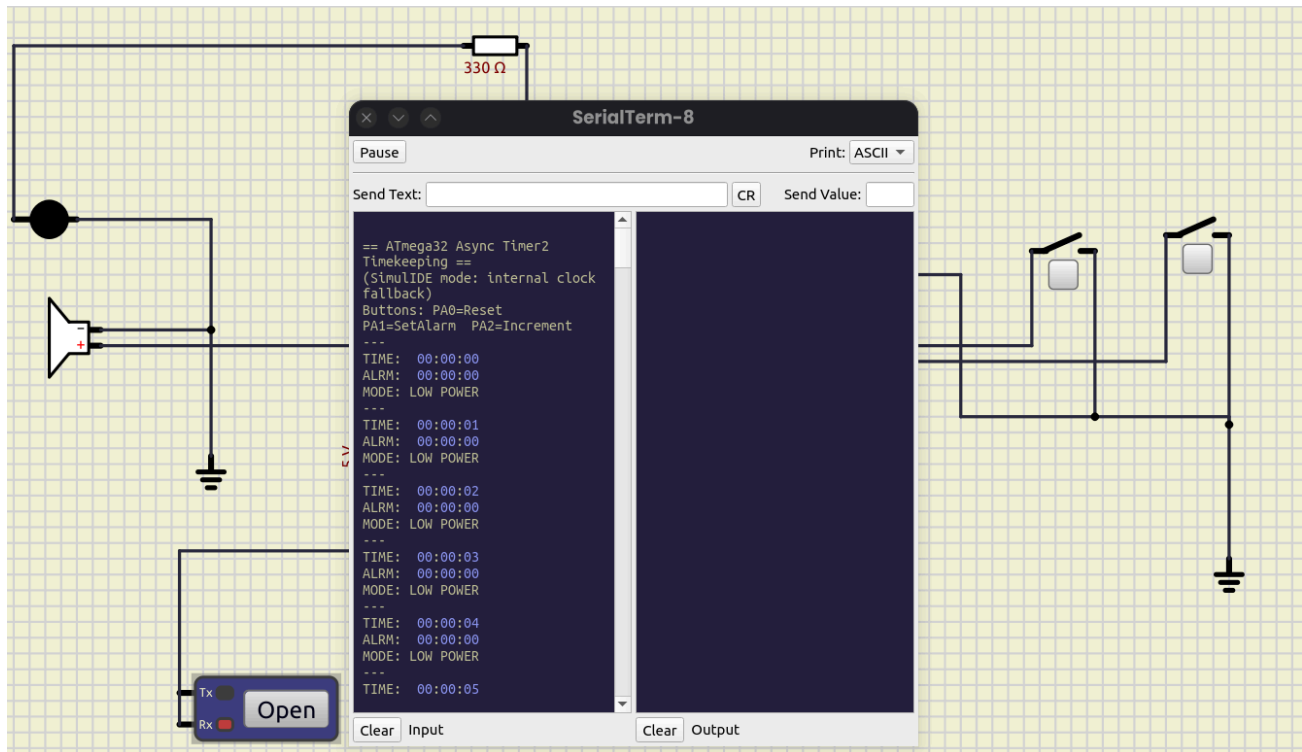


Figure 5.3 — Serial terminal showing seconds incrementing correctly in real time

TC-03: Wake-Up LED Toggles at 10-Second Intervals

Step	Action	Expected Result	Observed Result	Status
1	Observe LED symbol at PB0	LED toggles every 10 seconds	LED state changes at :00, :10, :20, :30... seconds	✓ PASS
2	Confirm via PORTB	PB0 XOR on every WAKEUP_INTERVAL multiple	Confirmed by LED indicator state in circuit	✓ PASS

Table 5.4 — TC-03 Results

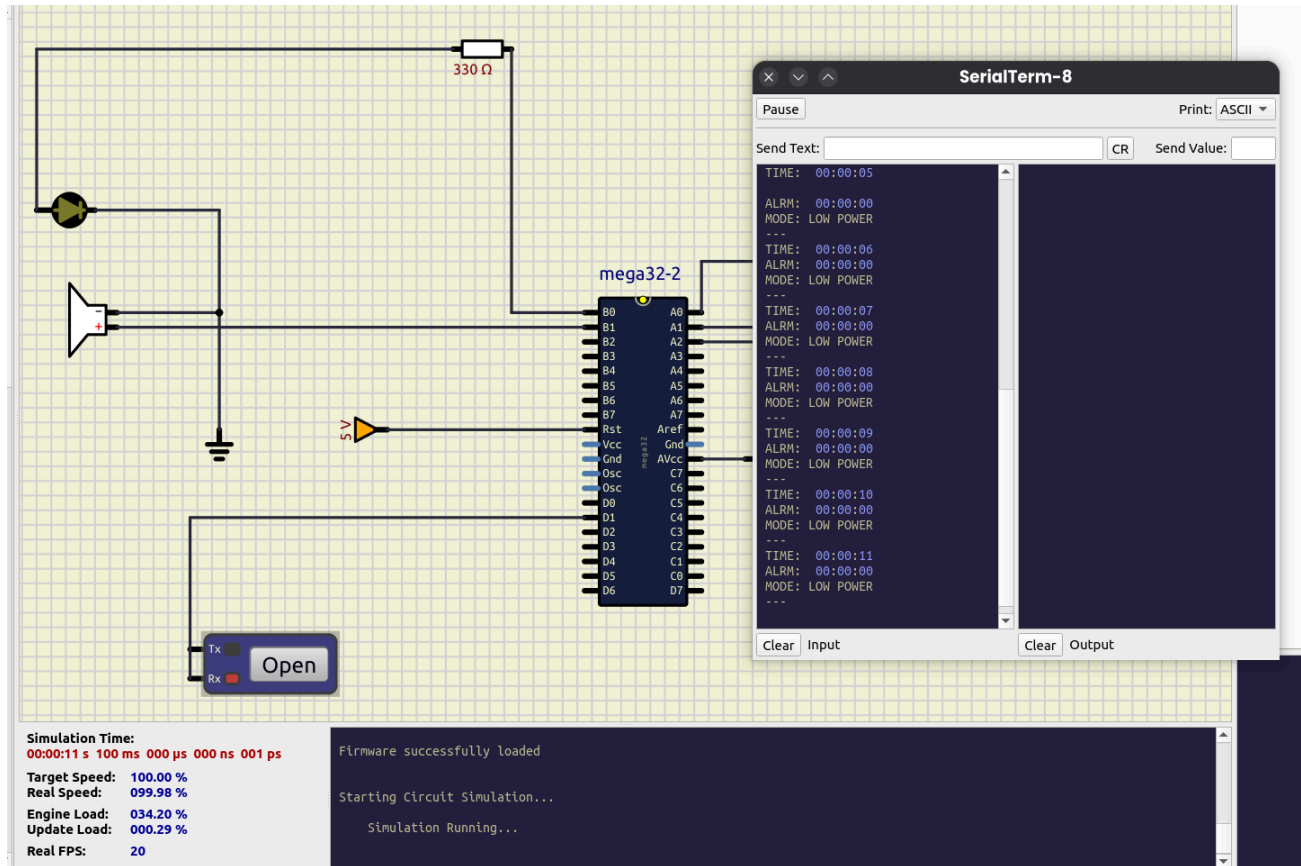


Figure 5.4 — LED indicator showing active state at a 10-second wake-up mark

TC-04: Alarm — Buzzer Activates at Programmed Time

Step	Action	Expected Result	Observed Result	Status
1	Press PA1 to enter SET ALARM HH mode	Serial shows MODE: SET ALARM HH	MODE field updated correctly	✓ PASS
2	Press PA2 to increment alarm hours	ALRM HH field increments	Alarm hours increment confirmed in ALRM display	✓ PASS
3	Press PA1 twice to reach SET ALARM SS, set seconds	Alarm field cycles correctly	All three fields (HH/MM/SS) settable and confirmed	✓ PASS
4	Press PA1 to exit to MODE_CLOCK; wait for alarm time	Buzzer activates for 3 seconds	Buzzer symbol activates; deactivates after 3 seconds	✓ PASS

Table 5.5 — TC-04 Results

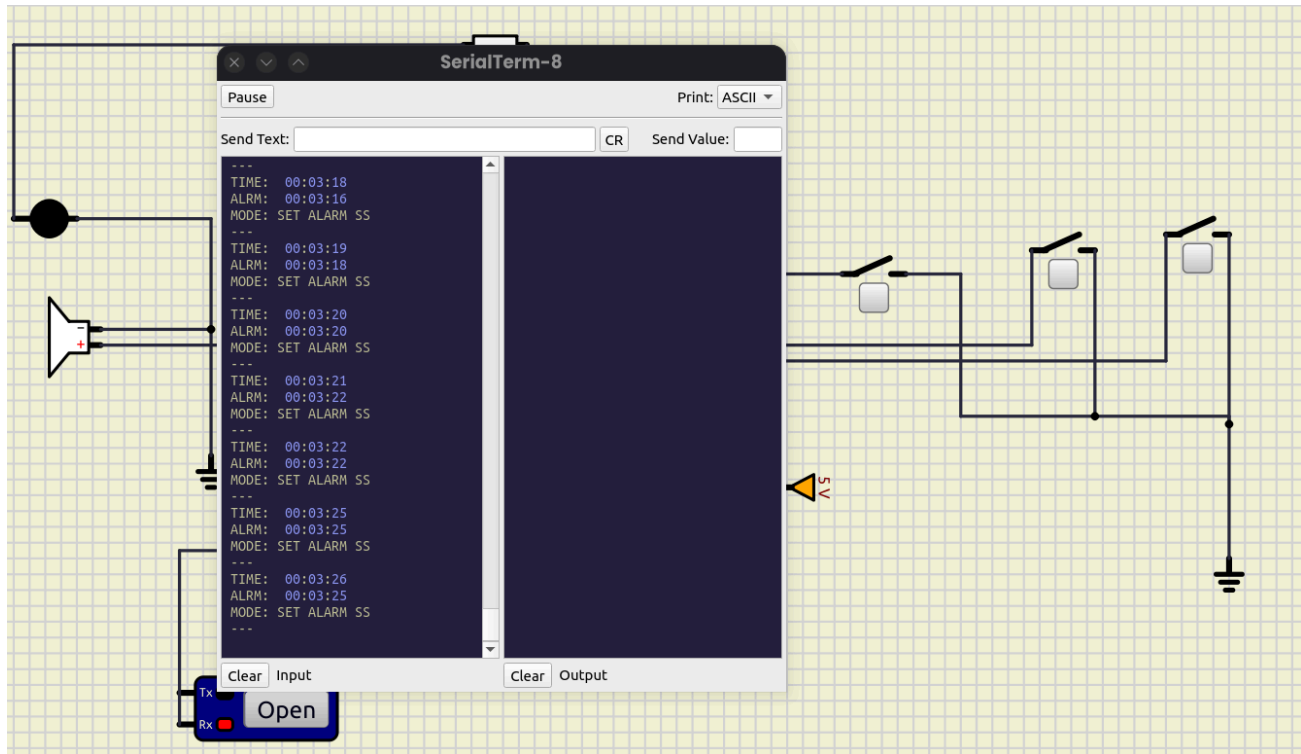


Figure 5.5 — Serial terminal in SET ALARM mode and buzzer active at alarm time

TC-05: Reset Button — Clock Returns to 00:00:00

Step	Action	Expected Result	Observed Result	Status
1	Press PA0 while clock is running	TIME resets to 00:00:00; MODE_CLOCK restored	TIME RESET TO 00:00:00 printed; time restarts	✓ PASS
2	Confirm alarm state	Alarm ringing cleared; buzzer stops if active	Buzzer and alarm state cleared on reset	✓ PASS

Table 5.6 — TC-05 Results

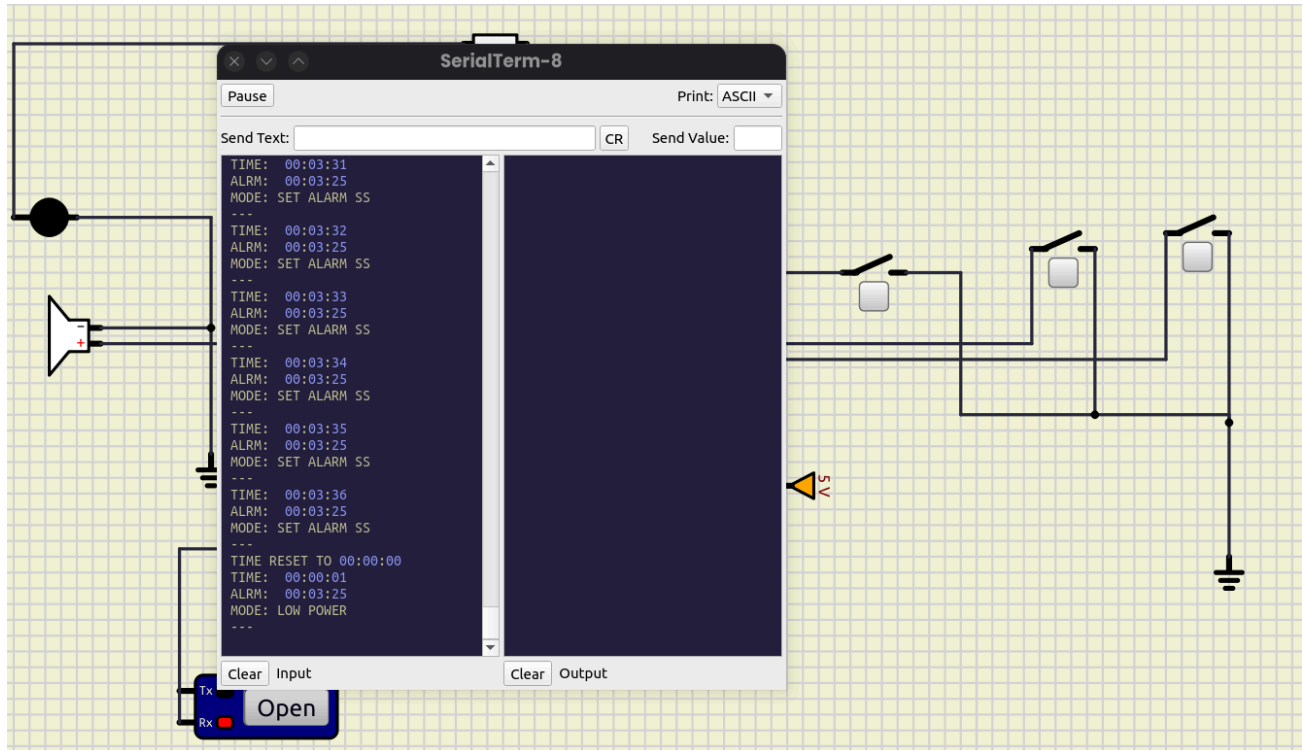


Figure 5.6 — Serial terminal showing *TIME RESET TO 00:00:00* message

TC-06: Sleep Mode Behaviour (Documented Limitation)

Step	Observation	Explanation	Status
1	Warning: "AVR SLEEP instruction not Fully implemented" in SimulIDE console	SimulIDE does not fully model Power-Save or Idle sleep modes	Documented
2	In SIMULIDE_MODE: SLEEP_MODE_IDLE is used instead of SLEEP_MODE_POWER_SAVE	IDLE keeps the synchronous clock running so Timer2 CTC continues ticking	Correct per design
3	On real hardware: SLEEP_MODE_POWER_SAVE would be used; async Timer2 continues from its own crystal while CPU is halted	Power-Save is the optimal mode for async Timer2 timekeeping; this is correct for the hardware design	Correct per design

Table 5.7 — TC-06: Sleep mode documentation

Chapter Six

Conclusion

6.1 Project Summary

This project successfully designed and implemented a low-power real-time timekeeping system on the ATmega32 microcontroller, centred on the asynchronous operation of Timer/Counter2 driven by a 32.768 kHz external watch crystal. The system achieved all required behaviours: accurate one-second timekeeping via Timer2 overflow interrupts, a ten-second wake-up LED heartbeat, a user-programmable buzzer alarm, software-debounced button input for clock reset and alarm configuration, and USART serial output displaying real-time clock state at 9600 baud.

The complete design was documented through a KiCad schematic with all required net labels and crystal placement, a PCB layout following the oscillator design rules specified in the project brief, and firmware verified in SimulIDE using a documented internal-clock fallback that transparently substitutes for the TOSC oscillator the simulator cannot model. The key technical challenge encountered — that Power-Save sleep mode halts the synchronous clock domain and therefore stalls a synchronously-clocked Timer2 fallback — was identified, explained, and resolved by switching to `SLEEP_MODE_IDLE` in the simulator path.

6.2 Learning Outcomes and Discussion

This project provided direct experience with several important embedded systems concepts:

- Asynchronous peripheral operation — understanding that Timer2's ability to run from a separate clock domain, decoupled from the main system clock, is a hardware feature unique to this peripheral among the ATmega32's three timers, and that this decoupling is the enabler of proper low-power timekeeping.
- Clock domain crossing — the mandatory register synchronisation wait loop in `timer2_init()` demonstrated that writing registers to hardware in a different clock domain requires explicit busy-wait acknowledgement; skipping this step silently corrupts the configuration.
- Power-Save sleep mode mechanics — learning that `SLEEP_MODE_PWR_SAVE` specifically preserves the asynchronous Timer2 oscillator domain while stopping

everything else. Using it with a synchronously-clocked Timer2 is a silent correctness bug: the timer stops, the wake-up source disappears, and the CPU parks indefinitely.

- USART baud rate sensitivity — discovering that a clock frequency mismatch between the firmware (F_CPU macro) and the simulator MCU configuration directly corrupts serial output, and that the fix is precise agreement between these two values.
- SimulIDE limitations and workarounds — developing the two-path firmware (real hardware vs. simulator fallback) as a template for working around simulator hardware gaps while keeping all application logic identical and testable.